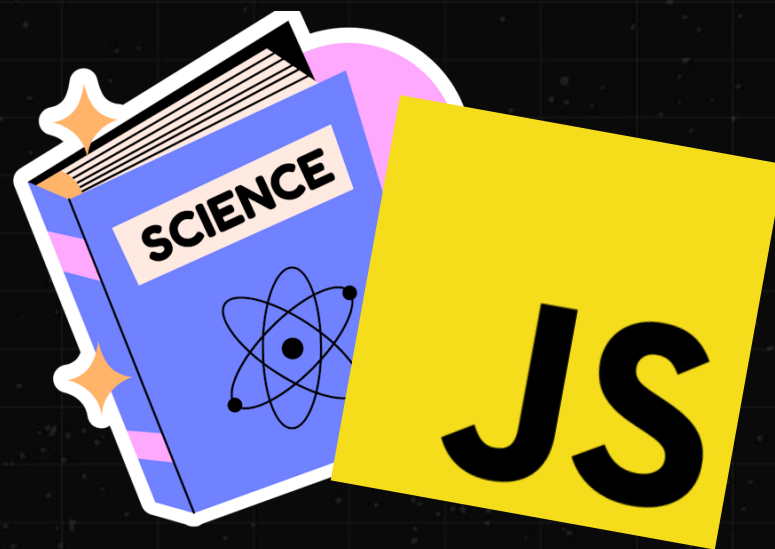


ЛЕКЦИЯ #2

# CS BO FRONTEND



# ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- \* Поговорить про **языки программирования**
- \* **Компиляция и интерпретация** программ
- \* Разобрать понятие **рантайма**
- \* **Типы и типизация** в языках программирования



# ПРОЦЕССОР ПОДДЕРЖИВАЕТ ФИКСИРОВАННЫЙ НАБОР ВОЗМОЖНЫХ ОПЕРАЦИЙ

- \* Как правило, это **достаточно примитивные операции**
- \* Например, сложить целое с другим целым
- \* Или **выполнить переход по адресу** (с условием или без)
- \* Полный список операций **можно найти в спецификации устройства**
- \* Каждой операции соответствует **отдельная команда – инструкция**
- \* **Любая компьютерная программа –**  
это просто последовательность инструкций

А КАК ПРОЦЕССОР  
ПОНИМАЕТ,  
КАКУЮ ИНСТРУКЦИЮ  
ВЫПОЛНЯТЬ?



# ВОТ СЕЙЧАС ВНИМАТЕЛЬНО

- \* Каждой инструкции соответствует некоторое число – опкод
- \* Какое это конкретно число определяется **архитектурой и спецификацией** устройства
- \* Процессор **интерпретирует такие числа** как команды к действию



НО КАК ОН ПОНИМАЕТ,  
ЧТО ПЕРЕД НИМ  
**ИНСТРУКЦИЯ,**  
А НЕ ЧИСЛО 0 ИЛИ  
ДРУГОЕ ЗНАЧЕНИЕ?



# ОН НЕ ПОНИМАЕТ

- \* Процессор может применить к **любому набору байт** любую из доступных операций
- \* А может **интерпретировать этот набор как инструкцию** и выполнить её
- \* Фактически, код программы – это **тоже данные**
- \* Наша программа состоит из **данных-кода (инструкций) и данных**

# КОГДА ПРОЦЕССОР ЗАГРУЖАЕТСЯ

- \* Он **читает некоторую область памяти** по статическому адресу
- \* Там должна находиться **самая первая инструкция**
- \* Процессор **выполнит её и перейдет к следующей** инструкции
- \* Инструкция может быть указанием, **где искать следующую инструкцию** (переход)
- \* В противном случае процессор будет **последовательно идти по памяти** и выполнять инструкции



# КОНЕЧНО МЫ УПРОЩАЕМ

- \* В реальной жизни включение компьютера содержит ряд дополнительных действий
- \* Например, **инициализация оперативной памяти**
- \* Первые инструкции процессоры занимаются загрузкой BIOS/UEFI
- \* Потом начинается загрузка операционной системы (если она есть)
- \* И так далее



# ЗАЧЕМ ВОООБЩЕ НУЖНЫ ЯЗЫКИ ПРОГРАММИРОВАНИЯ?

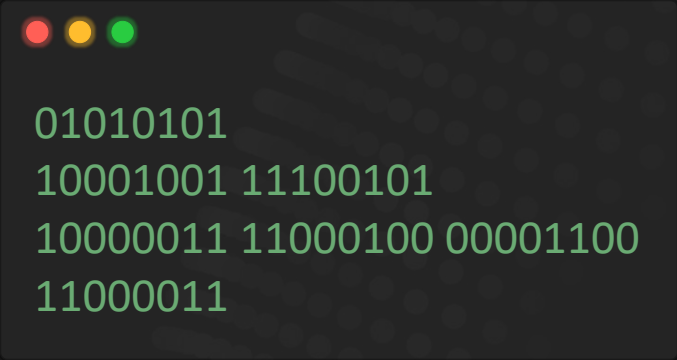




# «РОДНОЙ» ЯЗЫК КОМПЬЮТЕРА

- \* «Язык» устройства тесно связан с его физической моделью
- \* Электронные устройства используют бинарный язык – **«ТОК есть» и «тока нет»**
- \* С математической точки зрения это **можно представить как 0 и 1**
- \* Значит любая программа – это просто последовательность «этих цифр»
- \* Часть из них – **отвечает за операции**
- \* Часть – **за данные**

# НАПРИМЕР

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. It contains four lines of binary code in a light green monospace font.

```
01010101  
10001001 11100101  
10000011 11000100 00001100  
11000011
```

# РЕФЛЕКСИРУЕМ

- \* Наша программа представляет собой поток двоичных чисел (бит) сгруппированных в байты (обычно 8 бит)
- \* Такое представление программы в «понятном» для неё виде называется машинным кодом (реже байткодом)
- \* Все это имеет смысл, только в случае **конкретного вычислителя или архитектуры**
- \* Понять, что здесь происходит для человека не просто
- \* А писать так программы – еще сложнее



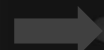
# НУЖНО УПРОЩАТЬ

- \* Возьмем шестнадцатеричную систему счисления
- \* Тогда записать любое число из 8 бит можно используя всего 2 буквы
- \* Например, 224 → 1110 0000 → e0
- \* Битам, которые отвечают за операции нужно дать **символические имена**

# НАПРИМЕР



```
push ebp
mov ebp, esp
add esp, 0xc
ret
```



```
01010101
10001001 11100101
10000011 11000100 00001100
11000011
```

# РЕФЛЕКСИРУЕМ

- ✱ Вместо того, чтобы писать на «родном» языке вычислителя – **мы придумали новый**
- ✱ **Этот язык понятнее людям**, но для вычислителя потребуется переводчик
- ✱ Программа, которая **преобразует текст на нашем языке в язык вычислителя**

МЫ СЕЙЧАС  
ГОВОРИМ ПРО  
АССЕМБЛЕР?





# ДА

- \* До появления языков ассемблера люди были вынуждены **использовать машинные коды**
- \* Человеку безумно **неудобно писать, читать и поддерживать** такой код
- \* Языки ассемблера позволяют дать машинному коду более **«человеческий» вид**
- \* Проблема в том, что **машина не «понимает» язык ассемблера**
- \* Нужно **преобразовать (транслировать) этот язык в машинные коды**
- \* Программа, которая делает это **называется ассемблером**

# ЧТО ДАЕТ ЯЗЫК АСЕМБЛЕРА

- \* **Вместо опкодов команд процессора** можно писать символические имена (mov eax, add acc и так далее)
- \* Для записи чисел можно использовать **более удобные системы счисления** (8, 10, 16)
- \* **Поддержка литералов строк** (обычно используется ASCII)
- \* Инструменты устранения дублирования кода
- \* Возможности могут отличаться для разных ассемблеров

|          |               |                                |             |
|----------|---------------|--------------------------------|-------------|
| 0089814E | 57            | PUSH EDI                       |             |
| 0089814F | 6A 00         | PUSH 0                         |             |
| 00898151 | FF95 57040000 | CALL DWORD PTR SS:[EBP+457]    | GlobalAlloc |
| 00898157 | 8938          | MOV DWORD PTR DS:[EAX],EDI     |             |
| 00898159 | 8D78 04       | LEA EDI,[EAX+4]                |             |
| 0089815C | 8D85 77040000 | LEA EAX,[EBP+477]              |             |
| 00898162 | 57            | PUSH EDI                       |             |
| 00898163 | 50            | PUSH EAX                       |             |
| 00898164 | E8 46000000   | CALL Decompress                |             |
| 00898169 | 57            | PUSH EDI                       |             |
| 0089816A | E8 EB000000   | CALL PrepareImage              |             |
| 0089816F | 89C6          | MOV ESI,EAX                    |             |
| 00898171 | 31C0          | XOR EAX,EAX                    |             |
| 00898173 | 83EF 04       | SUB EDI,4                      |             |
| 00898176 | 8B0F          | MOV ECX,DWORD PTR DS:[EDI]     |             |
| 00898178 | 57            | PUSH EDI                       |             |
| 00898179 | F3:AA         | REP STOS BYTE PTR ES:[EDI]     |             |
| 0089817B | 5F            | POP EDI                        |             |
| 0089817C | 57            | PUSH EDI                       |             |
| 0089817D | FF95 5B040000 | CALL DWORD PTR SS:[EBP+45B]    |             |
| 00898183 | 8BBD 67040000 | MOV EDI,DWORD PTR SS:[EBP+467] |             |
| 00898189 | 31C0          | XOR EAX,EAX                    |             |
| 0089818B | 83EF 04       | SUB EDI,4                      |             |
| 0089818E | 8B0F          | MOV ECX,DWORD PTR DS:[EDI]     |             |
| 00898190 | 57            | PUSH EDI                       |             |
| 00898191 | F3:AA         | REP STOS BYTE PTR ES:[EDI]     |             |
| 00898193 | 5F            | POP EDI                        |             |
| 00898194 | 57            | PUSH EDI                       |             |
| 00898195 | FF95 5B040000 | CALL DWORD PTR SS:[EBP+45B]    |             |
| 0089819B | 85F6          | TEST ESI,ESI                   |             |
| 0089819D | 74 07         | JE SHORT 008981A6              |             |
| 0089819F | 897424 1C     | MOV DWORD PTR SS:[ESP+1C],ESI  |             |
| 008981A3 | 61            | POPAD                          |             |
| 008981A4 | FFE0          | JMP EAX                        |             |
| 008981A6 | 61            | POPAD                          |             |
| 008981A7 | B8 FFFFFFFF   | MOV EAX,-1                     |             |
| 008981AC | C2 0800       | RETN 8                         |             |

# ВАЖНО ПОНИМАТЬ

- \* Язык ассемблера транслируется 1 к 1 в машинный код под конкретную архитектуру
- \* Это просто текстовое представление для машинного кода
- \* Языков ассемблера существует много и для каждого нужен свой ассемблер
- \* Разные ассемблеры могут отличаться поддерживаемыми возможностями
- \* Например, для x86 есть [NASM](#), [MASM](#), [FASM](#) и много других



# ПЛЮСЫ И МИНУСЫ ЯЗЫКОВ АССЕМБЛЕРА

- \* Это **очень низкий уровень абстракции**: ни переменных, ни типов, ни функций
- \* Под каждую новую архитектуру **нужно писать новый код**
- \* С другой стороны, программа написанная на языке ассемблера **потенциально будет самой эффективной из возможных**
- \* При условии, что вы очень хороший программист 😊

АССЕМБЛЕР СОЗДАЁТ  
ИЗ ИСХОДНОГО КОДА  
ФАЙЛ, КОТОРЫЙ  
МОЖНО ЗАПУСТИТЬ?

# ДА, НО ТУТ ЕСТЬ НЮАНСЫ

- \* Ассемблер действительно **может создать «исполняемый» файл**
- \* Например, .exe файл для архитектуры x86
- \* Только мы **крайне редко** запускаем программу на «голом» компьютере
- \* В большинстве случаев мы делаем это **в рамках некоторой операционной системы**
- \* К слову, .exe – это формат исполняемых файлов Windows

# ОС СОЗДАЕТ АБСТРАКЦИЮ НАД ЖЕЛЕЗОМ

- \* Разработчик работает с **программным интерфейсом ОС**, а не компьютером напрямую
- \* Это позволяет **значительно упростить разработку**
- \* **Добиться многозадачности** (в рамках одной ОС «одновременно» может работать много программ)
- \* И **повысить безопасность**
- \* Но **каждая ОС вводит свой формат исполняемых файлов**
- \* У Windows свой, у Mac OS свой, у Linux свой и так далее



ВЫХОДИТ,  
ЧТО ПРОЦЕССОР  
НЕ НАПРЯМУЮ  
ИСПОЛНЯЕТ  
ТАКОЙ ФАЙЛ?



# НЕТ

- \* **Исполнение программы занимается именно процессор**  
(используются инструкции конкретного вычислителя)
- \* ОС тут выступает в роли менеджера (процессы, разделение ресурсов)
- \* И подключаемой библиотеки (абстракция над железом)
- \* Чтобы ОС знала как начать исполнять файл –  
**он должен быть сформирован согласно заданной спецификации**
- \* Поэтому при трансляции ассемблером кода  
мы также **должны указать под какую ОС это сделать**

# НО ЭТО ЕЩЕ НЕ ВСЕ

- \* У разных версий одной и той же ОС **формат исполняемых файлов может измениться**
- \* Если ОС не предоставляет механизмов обратной совместимости, **то более старая программа может не запуститься на более новой ОС**
- \* Например, исполняемый файл от Windows 1-3 **просто не запустится** на Windows 11
- \* Для их работы потребуются системы виртуализации, которые искусственно **эмулируют одну ОС в рамках другой**
- \* Например, [DOSBox](#) для запуска MSDos приложений



А ДРУГИЕ ФАЙЛЫ  
ТОЖЕ ЧЕРЕЗ  
**АССЕМБЛЕР**  
ДЕЛАЮТСЯ?





# НЕТ

- \* Задача ассемблера транслировать исходный код текстового файла на языке ассемблера в **машинный код под конкретную архитектуру или ОС**
- \* На выходе получится либо **исполняемый файл**, либо файл с загружаемой библиотекой (тоже зависит от используемой ОС)
- \* Другие файлы, такие как .txt, .psd, .zip не являются исполняемыми и **для работы с ними нужна своя специальная программа**
- \* Например, .psd файлы создает Adobe Photoshop и он же нужен для их просмотра

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

# ВЫДЫХАЕМ

- \* Наша задача понять суть – это позволит **быстрее понять «что к чему»**, когда появится необходимость
- \* Например, когда вы начнете изучать WebAssembly
- \* Или если вы будете делать системы виртуализации, такие как [эмулятор архитектуры x86 на JS](#)
- \* Чтобы делать, например, такое <https://dos.zone>



# НО МЫ ТО ПИШЕМ НА JAVASCRIPT





# САМО СОБОЙ

- \* Языки ассемблера **слишком сложны и непереносимы**
- \* Поэтому люди стали придумывать **языки с более высоким уровнем абстракции**
- \* Разумеется, **процессор не умеет исполнять такие файлы**
- \* Как и в случае с языками ассемблера, нужна программа, которая **преобразует исходный код в машинный код**

# ЧТО НАМ ДАЛИ ЯЗЫКИ ВЫСОКОГО УРОВНЯ

- \* Добавляемые возможности **зависят от конкретного ЯП**, но есть общие черты
- \* Мы больше **не работаем с ячейками памяти напрямую**, а просто создаем переменные
- \* Появляются типы данных, которые **вводят строгие контракты** для разных видов информации
- \* Возможность в рамках одной программы **создать подпрограмму** для повторного использования (функции, процедуры)

# ПЕРЕНОСИМОСТЬ

- \* Языки высокого уровня уже **не привязаны к конкретному устройству**
- \* Мы можем написать транслятор в машинные коды **для разных устройств, архитектур и ОС**



```
// Главная функция, с исполнения которой стартует исполнение нашей программы
fn main() {
    // Локальные переменные внутри функции

    let a: f64 = 42.10; // Это значение в машинном коде будет закодировано как Double IEE 754
    let b: i32 = 10;    // Это значение в машинном коде будет закодировано как Int32
                        // используя прямой и дополнительный коды

    // Раз складываются два значения типа f64,
    // то в машинном коде будет использоваться инструкция сложения дробных чисел
    let sum = a + f64::from(b) /* Здесь вызывается "встроенная" в ЯП программа преобразования i32 к f64 */;

    // Выводим на экран данные в строковом виде:
    // у встроенного типа f64 есть стандартная логика преобразования в строку
    println!("a + b = {}", sum);
}
```



# КАЖДЫЙ ЯП ВВОДИТ СВОИ ПРАВИЛА ИГРЫ

- \* Какие парадигмы программирования поддерживаются (как мы пишем код)
- \* Какие есть встроенные типы и как они представлены в памяти
- \* Есть или нет **стандартная библиотека** и из чего она состоит
- \* И многое другое

# КАК ИСПОЛНЯЕТСЯ ПРОГРАММА НА ЯЗЫКЕ ПРОГРАММИРОВАНИЯ?



# ЕСТЬ НЕСКОЛЬКО ВАРИАНТОВ

- \* Можно создать программу, которая будет принимать исходный код и транслировать его в машинный под конкретную платформу
- \* Такой подход называется **компиляцией**, а такая программа – компилятором
- \* Например, так работает Rust или C/C++
- \* Помимо трансляции, компилятор может проверять ошибки в исходном коде и делать различные оптимизации, **чтобы конечный код работал эффективнее**

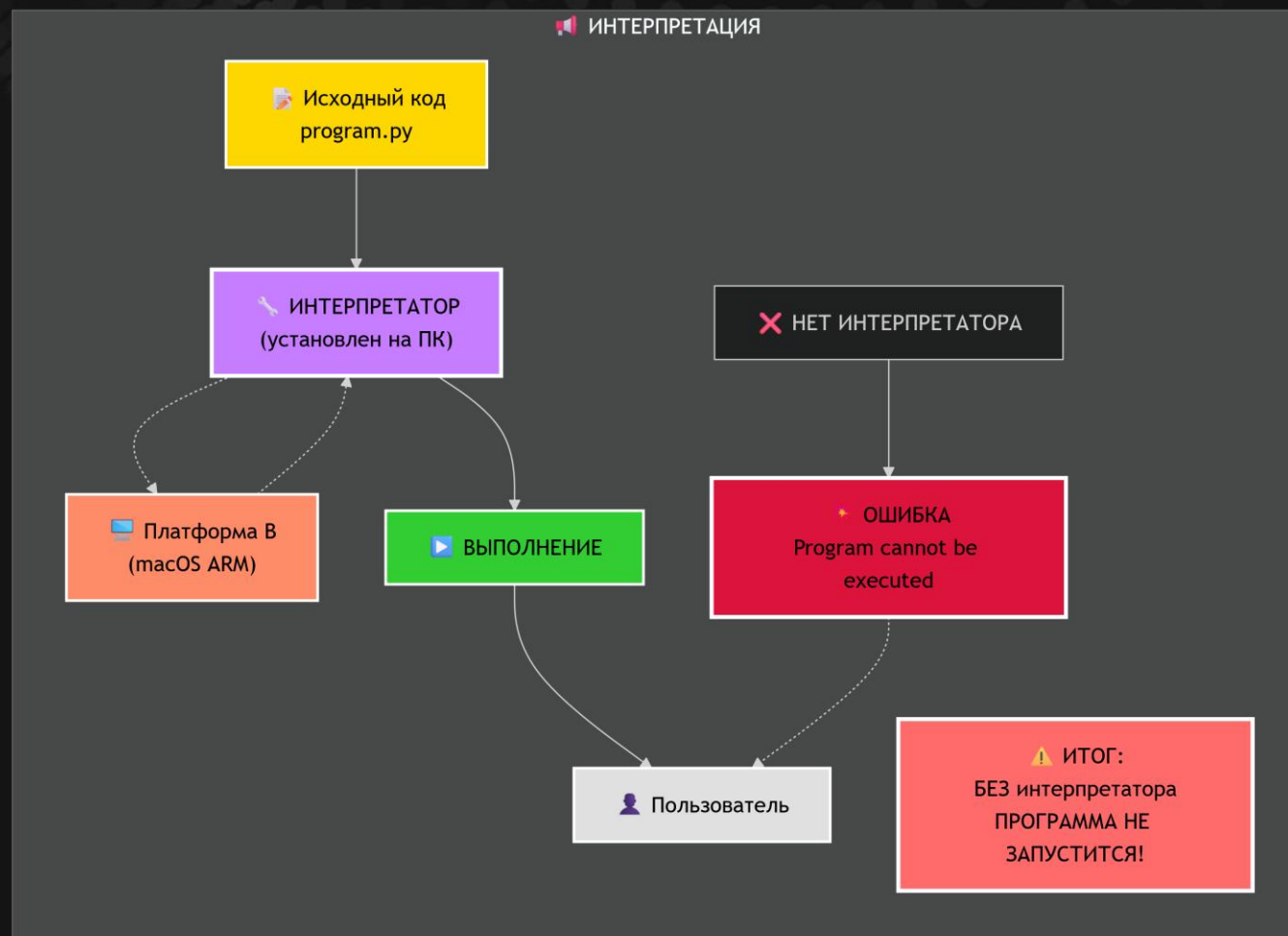
# ЯВНАЯ ФАЗА КОМПИЛЯЦИИ

- \* Компилятор **не исполняет файл с исходным кодом**, а преобразует его в то, что уже можно исполнить
- \* Это особенно важно, если наш компилятор **«умеет» оптимизировать код**
- \* Мы можем **«подождать подольше» в момент компиляции**, но зато конечная программа будет более эффективная
- \* По этой причине компилируемые ЯП часто отличаются **более высокой производительностью** исполнения конечного кода



# ЕСТЬ И ДРУГОЙ ВАРИАНТ

- \* Можно написать программу, которая принимает исходный код на некотором ЯП и начинаем немедленно его исполнять (интерпретировать)
- \* Такой подход называется **интерпретацией**, а такая программа интерпретатором
- \* Ключевая особенность в том, что нам теперь **не нужна явная фаза компиляции**



# ИНТЕРПРЕТАТОР ТРАНСЛИРУЕТ ИСХОДНЫЙ КОД В МАШИННЫЙ?



# НЕТ

- \* Интерпретатор – это программа, которая «понимает» код на исходном языке и **умеет его исполнять** на конкретном устройстве
- \* Сам интерпретатор – это программа, которая уже в свою очередь **может исполняться процессором «напрямую»** (это не всегда так)
- \* Если «интерпретатор» транслирует исходный код в машинный, а далее он исполняется уже самой машиной, то **это уже компиляция**
- \* Просто в этом случае фаза компиляции **делается неявно перед запуском программы**



```

function exec(code) {
  let variables = {};

  code.split(/\n;+/.forEach((op) => {
    let chunks = op.trim().split(/\s+/);

    switch (chunks[0]) {
      case "VAR": {
        if (!/[a-z]+/.test(chunks[1])) { throw new SyntaxError("Invalid variable name"); }
        variables[chunks[1]] = undefined;
        break;
      }

      case "SET": {
        if (!/[a-z]+/.test(chunks[1])) { throw new SyntaxError("Invalid variable name"); }
        if (!/\d+/.test(chunks[2])) { throw new SyntaxError("Invalid value to set"); }
        variables[chunks[1]] = parseInt(chunks[2]);
        break;
      }

      case "SHOW": {
        if (!/[a-z]+/.test(chunks[1])) { throw new SyntaxError("Invalid variable name"); }
        console.log(variables[chunks[1]]);
        break;
      }
    }
  }));
}

exec(`
  VAR a
  SET a 42
  SHOW a
`);

```

# РЕФЛЕКСИРУЕМ

- \* Мы разбили код на множество инструкций и **исполняли их один за другим**
- \* При этом каждый раз нам приходилось **«разбирать» исходный код** из формата строки в пригодный для работы вид
- \* Этот процесс называется **парсингом**

А НАСКОЛЬКО ЭТО  
ЭФФЕКТИВНО?



# ХОРОШИЙ ВОПРОС

- \* Наивная интерпретация **всегда будет значительно (на порядки) медленней** работы скомпилированной версии программы
- \* Поэтому такой подход применяется там, **где производительность не важна**
- \* Например, многие языки командной оболочки (Shell, Bash, ...) **являются простыми интерпретаторами**
- \* Если мы хотим сохранить гибкость интерпретируемого подхода, **но сделать его более эффективным**, то нужны улучшения



# ПРЕОБРАЗОВАНИЕ В БАЙТКОД

- \* Чтобы каждый раз не парсить инструкции из строки и проверять их корректность, можно **предварительно транслировать исходный код в удобный формат**
- \* Обычно используется **специально разработанный под задачу бинарный код**
- \* Такой код называют **байткодом**
- \* В отличие от машинного кода, **байткод нативно не поддерживается процессором**
- \* Для его исполнения **все равно нужен интерпретатор**
- \* Интерпретатор байткода называется **виртуальной машиной**



```
const VAR = 1;
const SET = 2;
const SHOW = 3;

function exec(code) {
  let variables = {};

  for (let ptr = 0; ptr < code.length; ptr++) {
    const op = code[ptr];

    switch (op) {
      case VAR: {
        variables[code[++ptr]] = undefined;
        break;
      }

      case SET: {
        variables[code[++ptr]] = code[++ptr];
        break;
      }

      case SHOW: {
        console.log(variables[code[++ptr]]);
        break;
      }
    }
  }
}

exec(new Uint32Array([
  VAR, "a".codePointAt(0),
  SET, "a".codePointAt(0), 42,
  SHOW, "a".codePointAt(0),
]));
```

# КЭШИРОВАНИЕ БАЙТКОДА

- \* Чтобы еще ускорить процесс интерпретации, можно «запомнить» единожды сгенерированный для файла байткод и **при последующих запусках брать сразу его**
- \* Такой подход называется **кэшированием байткода**
- \* Например, так делают большинство современных VM JavaScript в браузере
- \* Кэширование – это техника замены долгих вычислений за счет **сохранения и повторного использования ранее полученных результатов**

# ТУТ ЕСТЬ ЕЩЕ МОМЕНТ

- \* Когда у нас явная фаза компиляции, то **мы можем подождать**, пока компилятор с оптимизирует наш код
- \* Когда мы используем интерпретаторы, то обычно ожидается, что **исполнение начнется как можно быстрее**
- \* Поэтому многие тяжелые оптимизации просто не применимы
- \* Но существуют и такие интерпретаторы, которые стартует долго, но **потом работают быстрее**

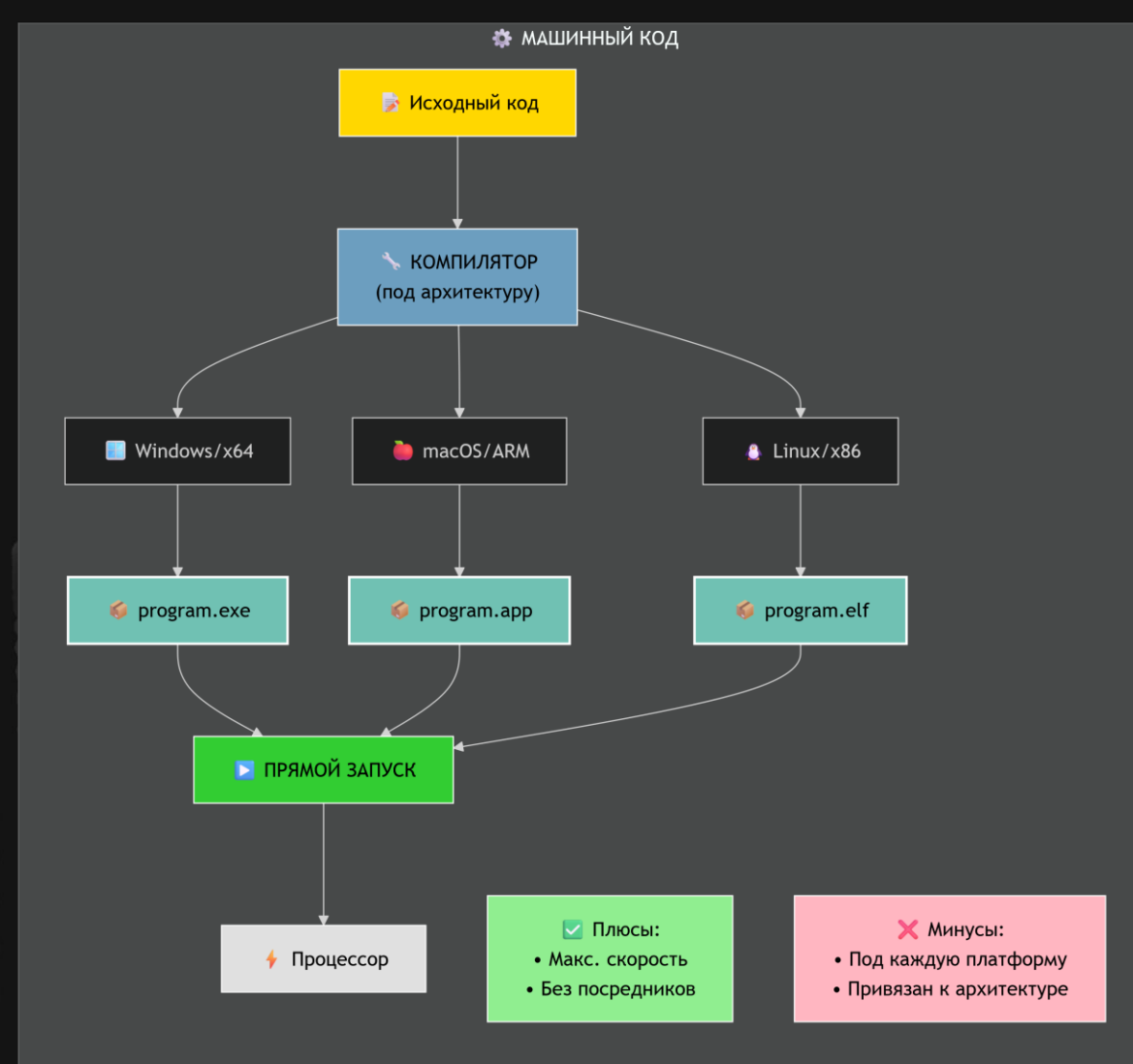


А НЕЛЬЗЯ ЛИ ЯВНО  
КОМПИЛИРОВАТЬСЯ  
В БАЙТКОД?



# РАЗУМЕЕТСЯ

- \* Компилятор **не обязан генерировать машинный код** под конкретную платформу
- \* Вместо этого он может **производить универсальный байткод**, который потом будет исполняться виртуальной машиной
- \* В таком случае можно делать больше оптимизаций на фазе компиляции, а при исполнении **больше нет смысла делать парсинг исходного кода**
- \* Такой подход использует Java или C#
- \* **Опционально** поддерживается многими интерпретаторами
- \* Например, CPython (стандартная реализация Python)



# В ЧЕМ ТУТ СОЛЬ

- \* Компиляция в байткод позволяет уйти от необходимости **отдельных компиляций** под кучу платформ – один раз скомпилировал и работает везде
- \* Но **для исполнения нашей программы требуется виртуальная машина**
- \* Скомпилированная в машинный код программа **все равно будет сильно быстрее**



А НЕЛЬЗЯ ЛИ  
ТРАНСЛИРОВАТЬ  
БАЙТКОД  
В МАШИННЫЙ?



# ТАК И ДЕЛАЮТ

- \* Байткод позволяет избавиться от избыточных компиляций, но **работает медленней**
- \* Если же **преобразовать байткод в машинный во время работы программы**, то проблема уйдет, ведь нам больше не нужен интерпретатор
- \* Можно преобразовать весь байткод в машинный код **перед непосредственным исполнением программы** (AOT, Ahead of Time)
- \* Например, такой подход используется в WebAssembly
- \* Или **постепенно преобразовывать отдельные части** (JIT, Just in Time)
- \* Такой подход используется в Java/C#/JS/Python/...

# СЛОЖНОСТИ С ТЕРМИНОЛОГИЕЙ

- \* Строго говоря, **компиляция кода отдельной фазой** в байткод или машинный код – это все равно AOT компиляция (она делается ДО исполнения)
- \* Но есть разница между **«сразу скомпилировать в машинный код»**
- \* Или **«скомпилировать в байткод, а затем компилировать в машинный код перед непосредственным исполнением»**
- \* В первом случае мы работаем сразу быстро, но **должны компилироваться отдельно под кучу платформ**
- \* Во втором компиляция одна, но **при холодном старте «надо подождать»**

БАЙТ-КОД + АОТ  
(WASM)

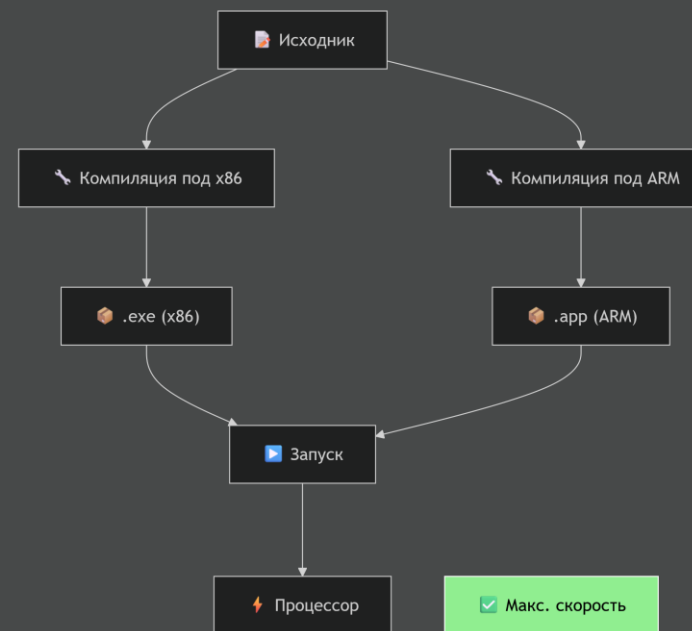


✓ Один код для всех

✗ Задержка при загрузке

✓ Сразу быстро

ПРЯМАЯ КОМПИЛЯЦИЯ



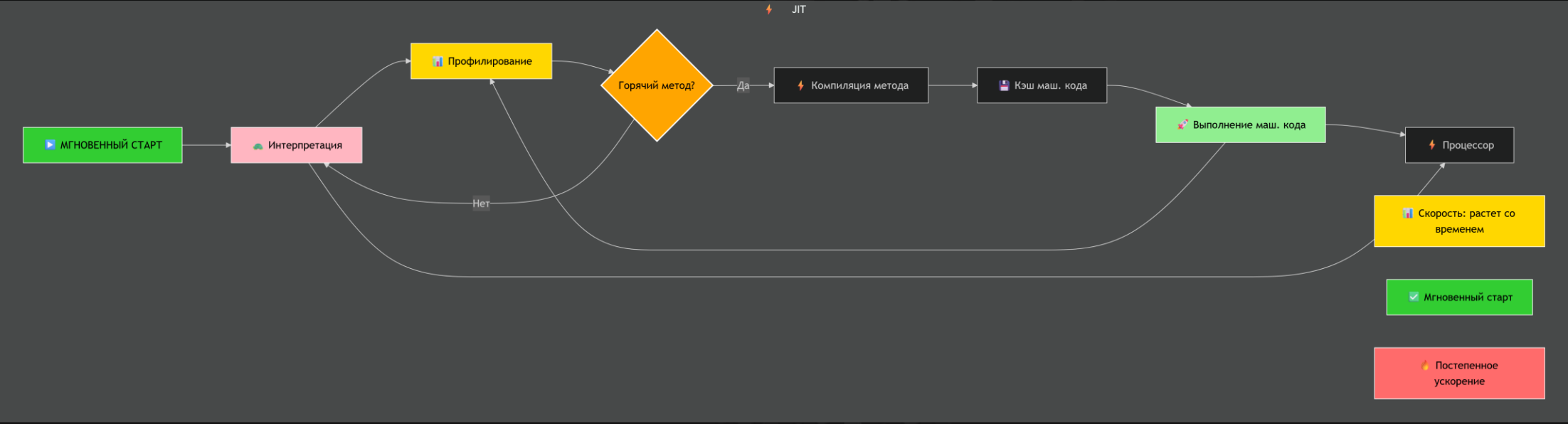
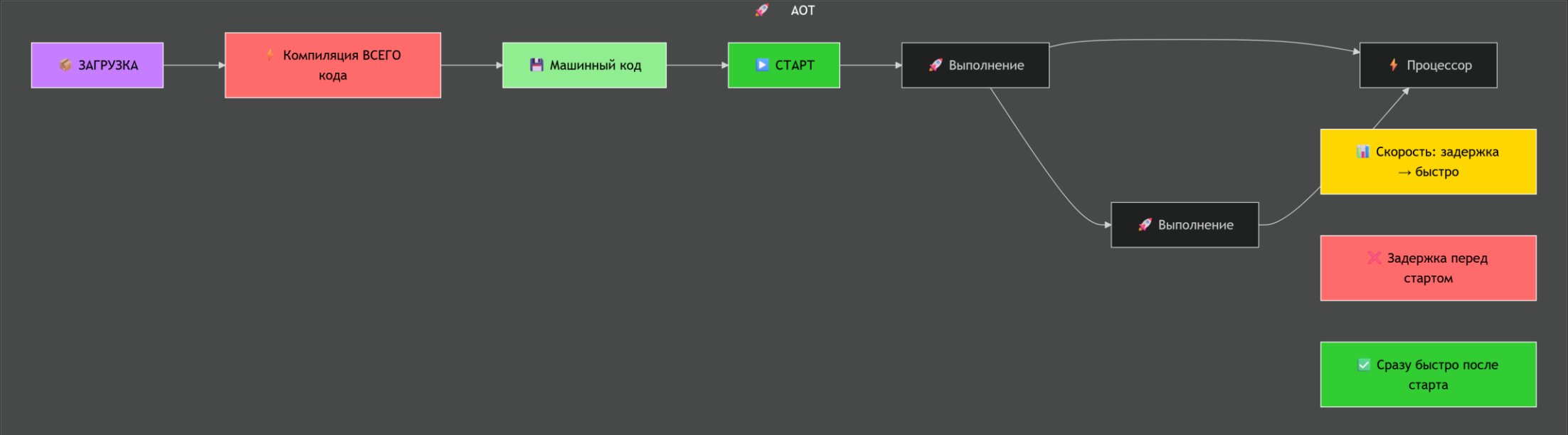
✓ Макс. скорость

✗ Под каждую платформу



# ОСОБЕННОСТЬ JIT

- \* JIT компилятор работает **параллельно с исполнением нашей программы**
- \* Программа **может стартовать сразу** работая через интерпретатор
- \* «Горячие» фрагменты или функции **постепенно компилируются в машинные кода**
- \* **Со временем** программа начинает работать быстрее
- \* С другой стороны, **можно специально «прогреть» отдельные части** – тогда получится что-то похожее на AOT



```

function measureTime(fn, iterations = 1000000, description = "") {
  const start = performance.now();

  for (let i = 0; i < iterations; i++) { fn(); }

  const end = performance.now();
  const totalTime = end - start;
  const avgTime = totalTime / iterations;

  console.log(`${description}:`);
  console.log(`Всего: ${totalTime.toFixed(3)} мс (${iterations.toLocaleString()} итераций)`);
  console.log(`Среднее: ${avgTime * 1000}.toFixed(3)} нс на итерацию`);
  console.log(`Операций/сек: ${Math.floor(iterations / (totalTime / 1000)).toLocaleString()}`);

  return { totalTime, avgTime };
}

function mathFn(x) {
  let result = 0;

  for (let i = 0; i < 10; i++) {
    result += Math.sin(x) * Math.cos(x) + Math.sqrt(Math.abs(x));
  }

  return result;
}

console.log("❌ ДО прогрева (интерпретация):");
const coldMath = measureTime(() => mathFn(42.5), 100000, " Холодный запуск");

console.log("\n✅ ПОСЛЕ прогрева (JIT-код):");
const hotMath = measureTime(() => mathFn(42.5), 100000, " Горячий запуск");

console.log("\n📈 Ускорение: ${coldMath.avgTime / hotMath.avgTime}.toFixed(2)}x");

```

❌ ДО прогрева (интерпретация):

Холодный запуск:

Всего: 3.455 мс (100 000 итераций)

Среднее: 0.035 нс на итерацию

Операций/сек: 28 946 911

✅ ПОСЛЕ прогрева (JIT-код):

Горячий запуск:

Всего: 2.125 мс (100 000 итераций)

Среднее: 0.021 нс на итерацию

Операций/сек: 47 047 753

📈 Ускорение: 1.63x

МЫ МОЖЕМ  
ИСКУССТВЕННО  
УСКОРЯТЬ НАШ  
КОД?





# ДА, НО ТУТ ЕСТЬ НЮАНСЫ

- \* Во-первых, в браузере такое делать – будет плохой идеей, ведь мы будем **«тормозить и фризить»** на старте
- \* А вот на сервере – это **обычная практика для сред с JIT** немного «прогреться» на старте
- \* Но в контексте JavaScript тут добавляются новые нюансы

# JIT И ДИНАМИЗМ JAVASCRIPT

- \* JIT в JS работает на уровне функций
- \* **Но не каждая JS функция** в принципе может быть скомпилирована JIT
- \* Например, если внутри функции используете `eval`,  
то **JIT никогда не будет её оптимизировать**
- \* Здесь работает правило: **чем строже код, тем больше шансов на успех**
- \* В JS функции могут принимать разные типы аргументов –  
JIT будет вынужден создавать **отдельные функции** для разных типов

```

function measureTime(fn, iterations = 1000000, description = "") {
  const start = performance.now();

  for (let i = 0; i < iterations; i++) { fn(); }

  const end = performance.now();
  const totalTime = end - start;
  const avgTime = totalTime / iterations;

  console.log(`${description}:`);
  console.log(`Всего: ${totalTime.toFixed(3)} мс (${iterations.toLocaleString()} итераций)`);
  console.log(`Среднее: ${avgTime * 1000}.toFixed(3)} нс на итерацию`);
  console.log(`Операций/сек: ${Math.floor(iterations / (totalTime / 1000)).toLocaleString()}`);

  return { totalTime, avgTime };
}

function mathFn(x) {
  let result = 0;

  for (let i = 0; i < 10; i++) {
    eval(`result += Math.sin(x) * Math.cos(x) + Math.sqrt(Math.abs(x))`);
  }

  return result;
}

console.log("❌ ДО прогрева (интерпретация):");
const coldMath = measureTime(() => mathFn(42.5), 100000, " Холодный запуск");

console.log("\n✅ ПОСЛЕ прогрева (JIT-код):");
const hotMath = measureTime(() => mathFn(42.5), 100000, " Горячий запуск");

console.log("\n📊 Ускорение: ${coldMath.avgTime / hotMath.avgTime}.toFixed(2)}x");

```

❌ ДО прогрева (интерпретация):  
 Холодный запуск:  
 Всего: 89.551 мс (100 000 итераций)  
 Среднее: 0.896 нс на итерацию  
 Операций/сек: 1 116 682

✅ ПОСЛЕ прогрева (JIT-код):  
 Горячий запуск:  
 Всего: 89.340 мс (100 000 итераций)  
 Среднее: 0.893 нс на итерацию  
 Операций/сек: 1 119 321

📊 Ускорение: 1.00x

# JIT СПЕЦИФИКУ НУЖНО УЧИТЫВАТЬ

- ✱ Если пишете бенчмарк, то **либо сравнивайте работу без JIT**
- ✱ Например, Node.js можно запустить с флагом `node --jitless`
- ✱ **Либо предварительно прогревайте** все тест кейсы

```
function measureTime(fn, iterations = 1000000, description = "") {  
  // Прогреваем функцию (чтобы избежать накладных расходов на первый запуск)  
  if (iterations > 1000) {  
    for (let i = 0; i < 1000; i++) fn();  
  }  
  
  // Основное измерение  
  // ...  
}
```



МОЖНО ЛИ  
УВИДЕТЬ, ЧТО JIT  
ПРИМЕНИЛСЯ?



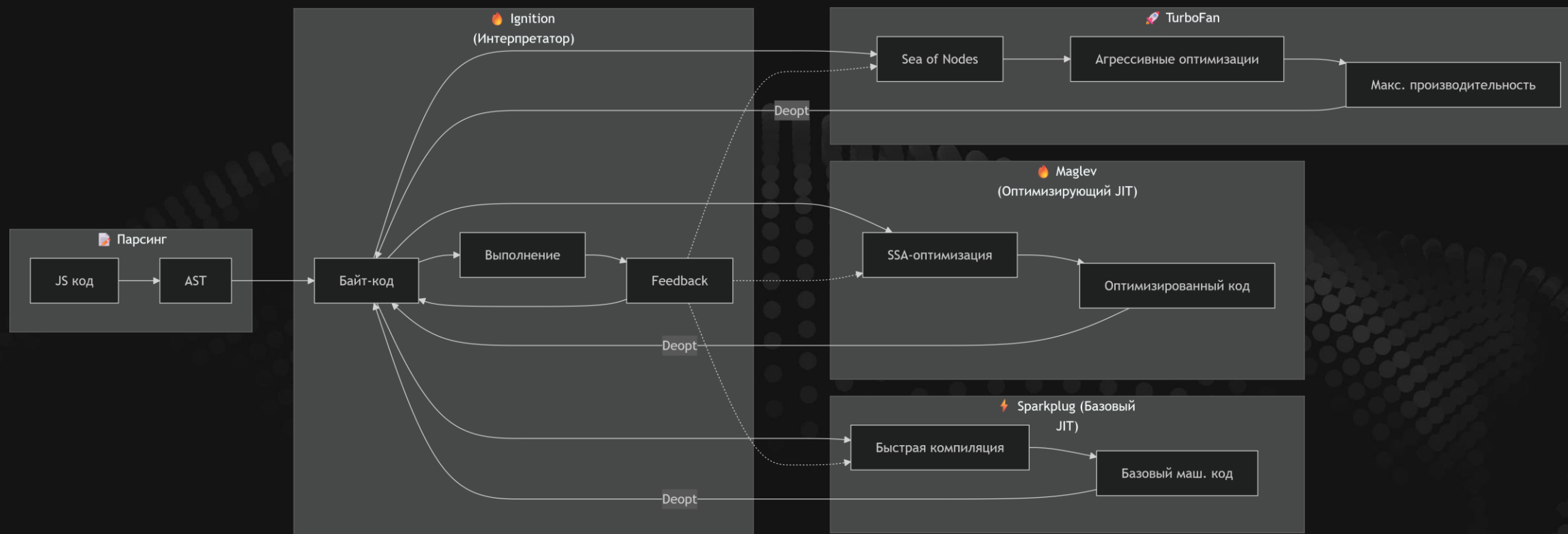
# ЗАВИСИТ ОТ ИНСТРУМЕНТОВ

- \* В браузерах в отладчике обычно есть вкладка «Производительность», где **можно исследовать такие вещи**
- \* В Node.js тоже можно использовать отладчик  
`node --inspect` или `node --inspect-brk`
- \* Node.js можно запустить с флагами `node --trace-opt --trace-deopt`
- \* Или флаги `node --prof` и `node --prof-process`

# В РЕАЛЬНОЙ ЖИЗНИ ВСЕ СЛОЖНЕЕ

- \* Наличие или отсутствие JIT компилятора никак не регламентируется в спецификации JavaScript
- \* Даже наличие байткода или его формат (а вот в Java есть [спека на байткод](#))
- \* Реализаций JavaScript существует очень много: [V8](#), [SpiderMonkey](#), [JavaScriptCore](#)
- \* Минималистичные реализации: [QuickJS](#), [JerryScript](#)
- \* Они следуют [спецификации ECMAScript](#), но нюансы реализации у всех свои

# НАПРИМЕР, V8





second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

# В ЧЕМ СОЛЬ

- \* JavaScript поставляется к исполнению в «исходных кодах» как текст
- \* Выполнение должно начинаться **как можно быстрее** (интерпретатор)
- \* Параллельно нужно начать делать как можно быстрее **простые оптимизации**
- \* Фоном можно применять **более «дорогие»**, но эффективные оптимизации
- \* В V8 для этого используется **3 слоя разных JIT компиляторов + интерпретатор**
- \* Схема может меняться в любой момент



ПОЛУЧАЕТСЯ  
ВИРТУАЛЬНАЯ МАШИНА –  
ЭТО НЕ ТОЛЬКО  
ИНТЕРПРЕТАТОР?

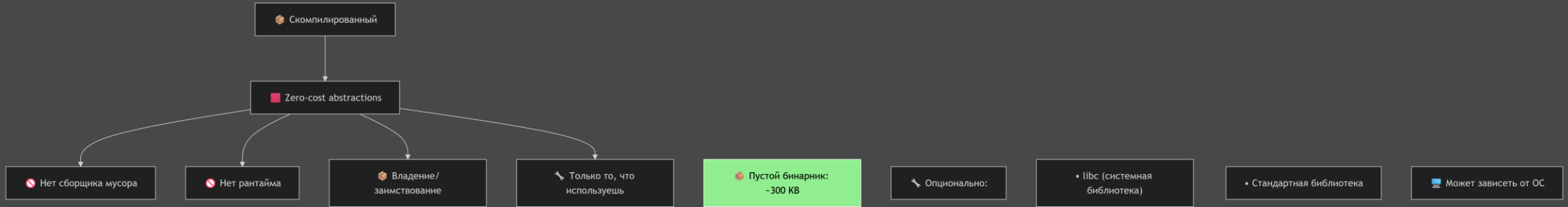


# В ТОЧКУ

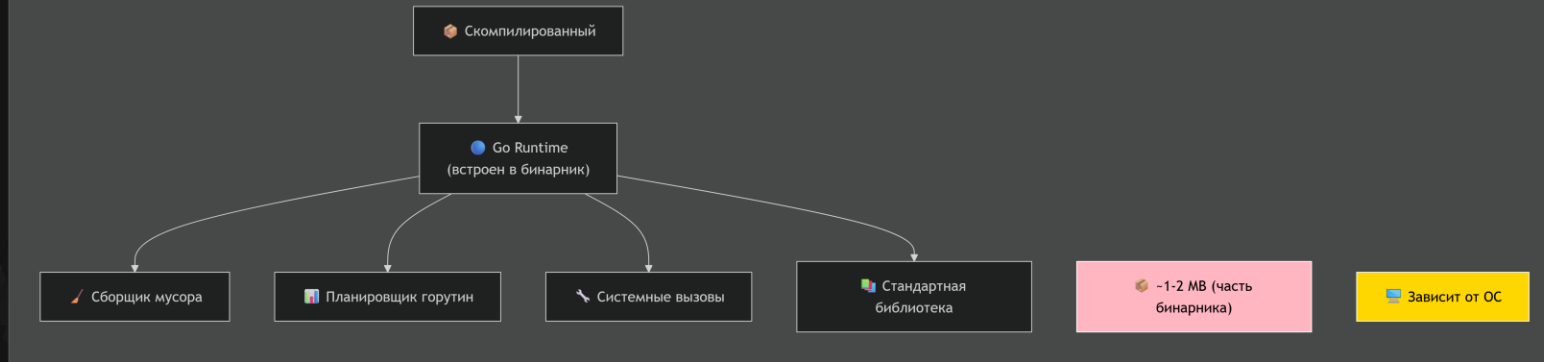
- \* Виртуальная машина – это **целая экосистема связанных инструментов**
- \* Интерпретатор, сборщик мусора, JIT компилятор и даже отладчик
- \* Без этого **наш код просто не выполниться**
- \* Все, что необходимо для работы нашей программы, но не является кодом называется рантаймом



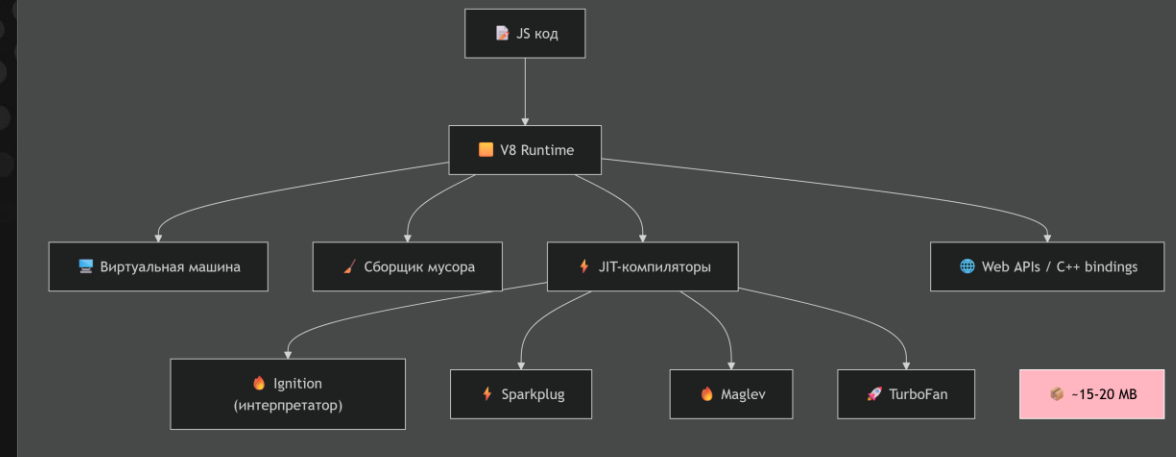
RUST



GOLANG



JAVASCRIPT



# РЕФЛЕКСИРУЕМ

- \* Рантайм – это как бы часть вашей программы необходимая для её работы
- \* Например, виртуальная машина или сборщик мусора
- \* Реализации разных ЯП отличаются размером своего рантайма
- \* Когда рантайм небольшой, то такой язык проще встроить в новое окружение
- \* Например, в WASM

# ЯЗЫКОВ ПРОГРАММИРОВАНИЯ ОЧЕНЬ МНОГО

- \* Языки отличаются как **дизайном** и **используемой парадигмой**
- \* Так и **способами работы с памятью** и **общим уровнем абстракции**
- \* Например, C или Rust можно назвать низкоуровневыми языками
- \* А JS или Python высокоуровневыми

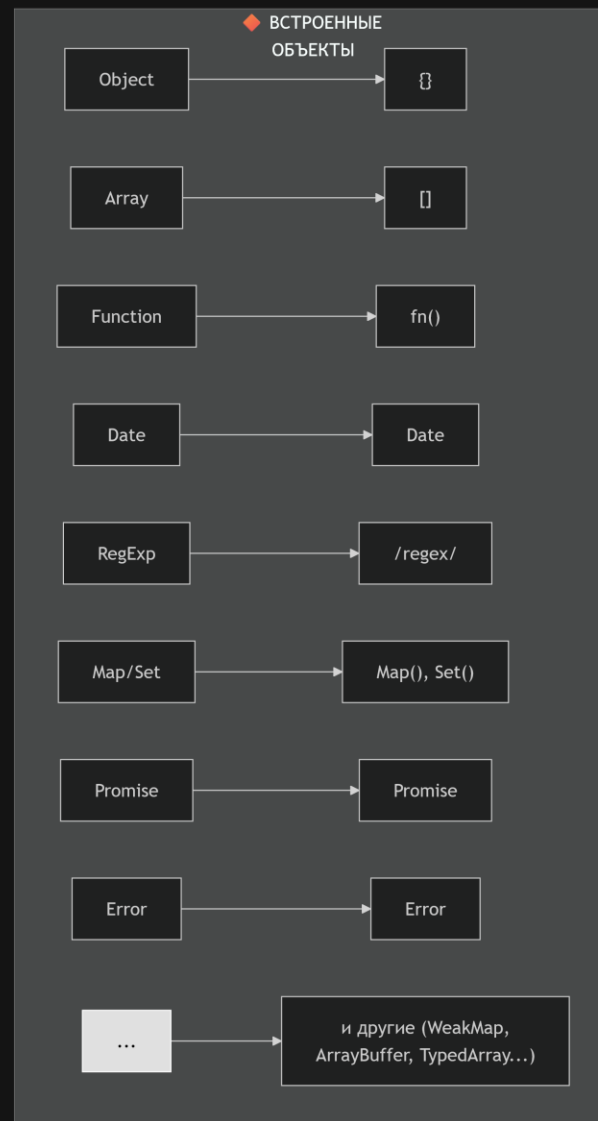
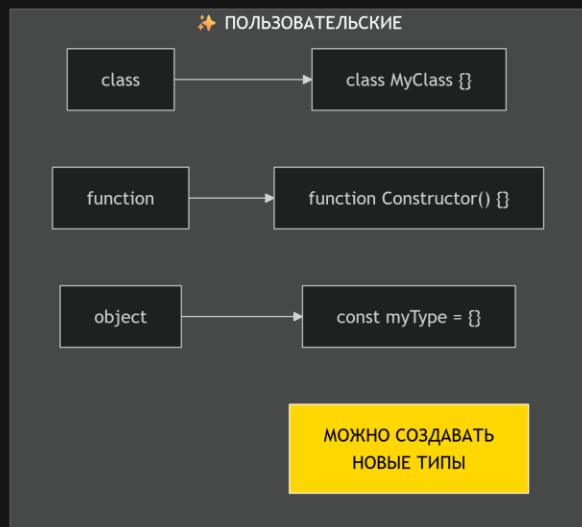
# ЯП И ТИПЫ ДАННЫХ

- \* Мы помним, что для процессора любые данные это поток байтов
- \* Мы можем ввести контракты над данными и работать с ними определенным образом
- \* ЯП могут ввести ряд правил для гарантии выполнения этих контрактов
- \* Система правил как можно работать с типами в рамках ЯП называется типизацией



# ВСТРОЕННЫЕ ТИПЫ

- ✱ ЯП зачастую вводят множество **предопределенных базовых типов**
- ✱ Количество таких типов, а также **способ их представления в памяти** зависит от ЯП
- ✱ Так и **от реализации этого ЯП**



📁 ТИПЫ JAVASCRIPT

♦ ПРИМИТИВЫ (7)

♦ ОБЪЕКТЫ

✦ ПОЛЬЗОВАТЕЛЬСКИЕ

А ЧТО ТАМ С  
ТИПИЗАЦИЕЙ?

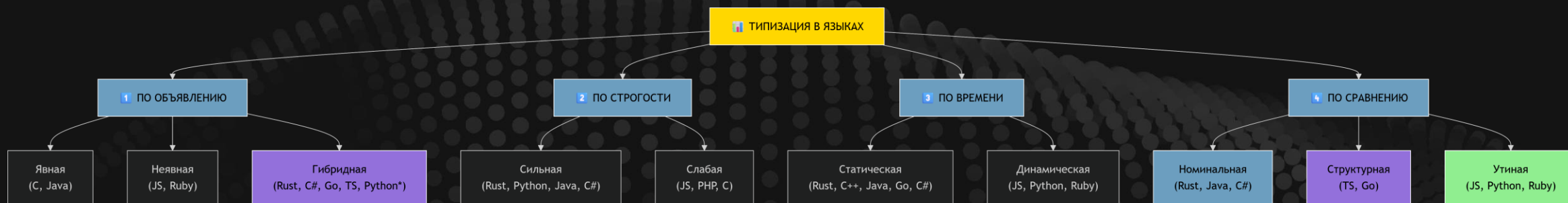


# КАКАЯ БЫВАЕТ ТИПИЗАЦИЯ

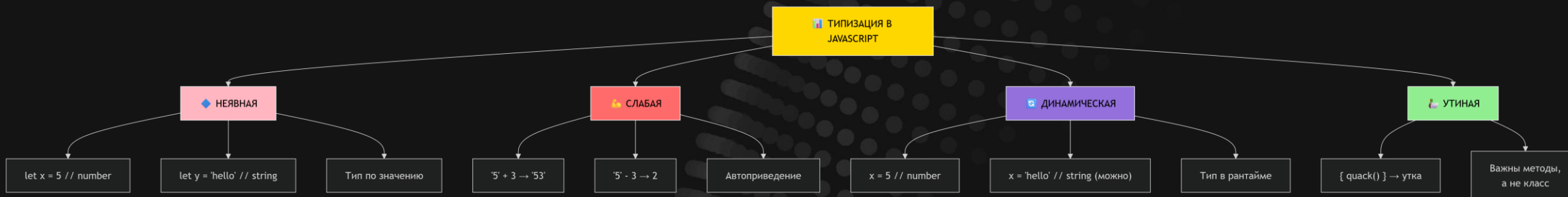
- \* Явная / неявная / гибридная
- \* Сильная (строгая) / слабая (нестрогая)
- \* Статическая / динамическая
- \* Структурная / номинальная / утиная



# СРАВНИМ



# ТИПИЗАЦИЯ JAVASCRIPT



# НА ЧТО ВЛИЯЕТ ТИПИЗАЦИЯ

- \* Количество ошибок которые можно отловить на этапе написания кода
- \* Простота написания кода
- \* Простота поддержки кода
- \* Эффективность работы кода
- \* Предсказуемость кода

# КОРОТКО О СЛАБОЙ ТИПИЗАЦИИ

```
>> '' == 0
```

```
← true
```

```
>> '0' == 0
```

```
← true
```

```
>> '0' == ''
```

```
← false
```

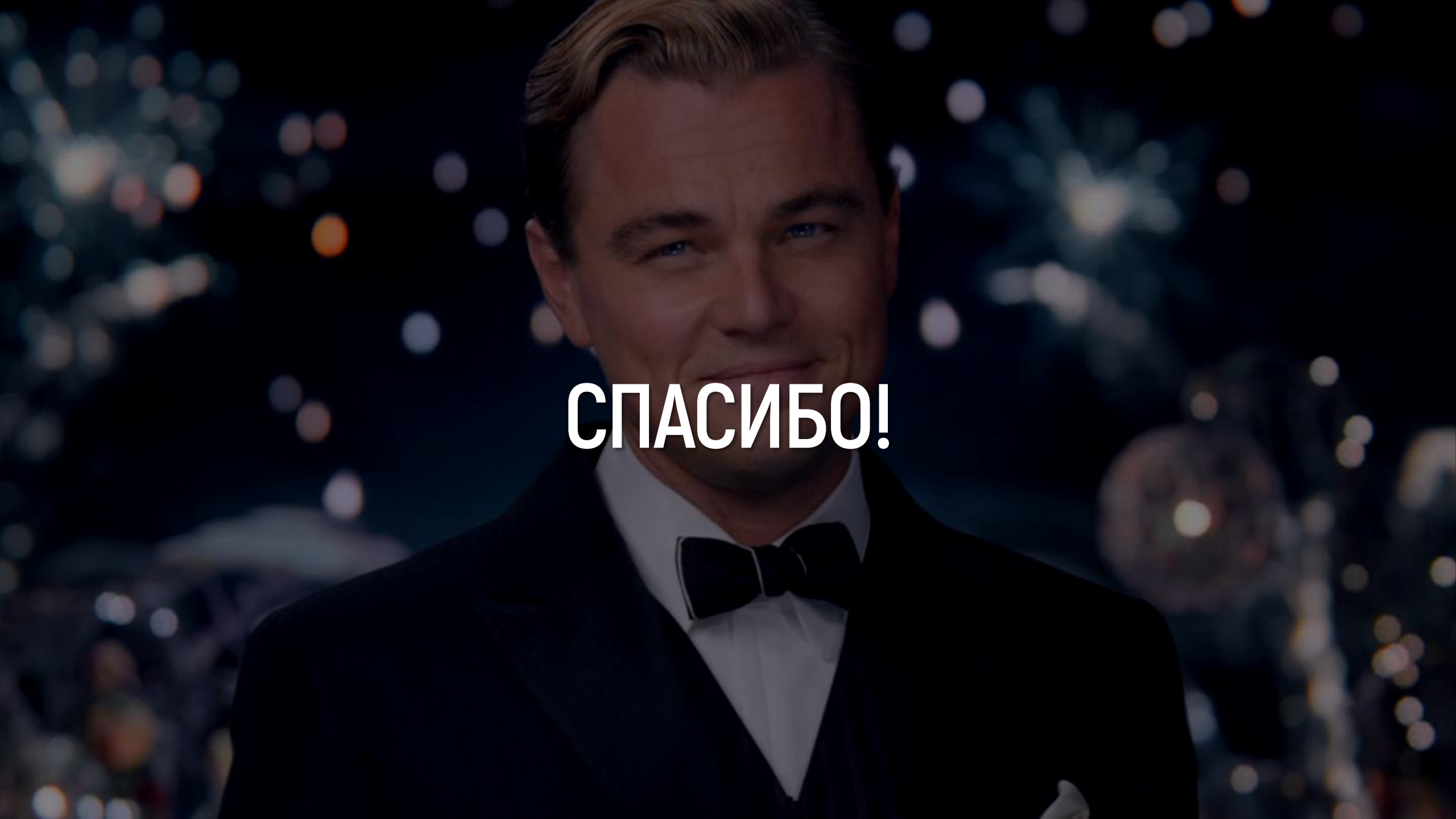


# ВЫБОР ТИПИЗАЦИИ ОПРЕДЕЛЯЕТ МНОГОЕ

- \* Статическая/гибридная/сильная – надежно и эффективно, но сложно
- \* Динамическая/неявная/слабая – просто и гибко, но не очень эффективно и надежно
- \* Динамическая/гибридная/сильная – просто, гибко и достаточно эффективно

# ПОДВЕДЕМ ИТОГИ

- \* Языки программирования **нужны людям**, чтобы абстрагировать от языка машины
- \* Языки программирования отличаются **не только дизайном, но и реализацией**
- \* Для исполнения JavaScript **необходима виртуальная машина**
- \* Современные виртуальные машины JS **активно используют JIT компиляцию**
- \* Но нюансы сильно **привязаны к конкретной версии VM**
- \* Важнейшая часть дизайна ЯП – это **то, как он работает с типами**
- \* JS использует **динамическую неявную слабую утиную типизацию**

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus bokeh lights in various colors (white, yellow, orange, blue), suggesting a night-time event or fireworks.

**СПАСИБО!**